JIMMA CITY ADMINISTRATION

Jimma City Smart Asset Management System

System and Architecture Design Document

Architecture, key components, data flows and design decisions

This document describes how the Jimma City Smart Asset Management System is built: the parts it is made of, how they are deployed, how data moves between users, the application, the database and outside services, and the reasons behind the main design choices. It is written so that the INSA assessment team can plan the security test of the web application and its API without first reading the source code.

Document number	JCSAMS-SDD-002
Version	1.0 (submission issue)
Date of issue	25 October 2026
System owner	Jimma City Administration, Ethiopia
Developed by	Banmen Technologies
Prepared by	Bealem Sintayehu, Project Lead, Banmen Technologies
Submitted to	Information Network Security Administration (INSA), through the Hibir platform
Assessment requested	Web application and API security assessment (penetration test)
Classification	RESTRICTED

Document control

Revision history

Version	Date	Author	Description of change
0.1	2 October 2026	Bealem Sintayehu	First outline: components and deployment.
0.5	18 October 2026	Bealem Sintayehu	Data flow diagrams, entity relationships, approval states added after the role review.
0.9	24 October 2026	Bealem Sintayehu	Updated for the security work: account lockout, idle timeout, server-side sign-out, temporary password expiry.
1.0	25 October 2026	Bealem Sintayehu	Issued for the INSA assessment.

Review and approval

This document is valid for submission once the people below have reviewed and signed it. Signatures are collected on the printed copy kept in the project file at the Jimma City Administration.

Role	Name	Position	Signature	Date
Prepared by	Bealem Sintayehu	Project Lead, Banmen Technologies		
Reviewed by		Head, ICT Directorate, Jimma City Administration		
Approved by		Office of the Mayor, Jimma City Administration		

Distribution

Copy	Recipient	Form
1	Information Network Security Administration (INSA), assessment team	Electronic, through the Hibir platform
2	Office of the Mayor, Jimma City Administration	Printed and electronic
3	ICT Directorate, Jimma City Administration	Printed and electronic
4	Project file	Printed, signed original

Related documents in this submission

Number	Title
JCSAMS-LTR-000	Covering letter to INSA
JCSAMS-SRS-001	Software Requirements Specification
JCSAMS-SDD-002	System and Architecture Design Document
JCSAMS-TDD-003	Technical Design and Specification
JCSAMS-UG-004	User Guide
JCSAMS-SEC-005	Security Documentation

Contents

1 Introduction . . . 5

1.1 Purpose of this document . . . 5

1.2 Intended readers . . . 5

1.3 Scope . . . 5

1.4 Definitions and abbreviations . . . 5

1.5 References . . . 6

2 System overview . . . 6

2.1 What the system does . . . 6

2.2 Users and their areas . . . 7

2.3 System context . . . 7

3 Architecture . . . 9

3.1 Architectural style . . . 9

3.2 Architecture diagram . . . 9

3.3 Client tier . . . 10

3.4 Application tier . . . 11

3.5 Data tier . . . 11

4 Key components . . . 12

4.1 Component dependencies . . . 13

4.2 Third-party components . . . 13

5 Deployment architecture . . . 14

6 Data flow design . . . 15

6.1 Context level (level 0) . . . 15

6.2 Main processes (level 1) . . . 15

6.3 Asset registration (level 2 of process 2.0) . . . 18

6.4 Data flow explanation . . . 18

6.5 Sequence of a sign-in and an authenticated request . . . 20

6.6 Sequence of a QR label scan . . . 20

6.7 AI assistant . . . 21

7 Data design . . . 21

7.1 Core entities and relationships . . . 21

7.2 Design rules for data . . . 22

7.3 Data classification . . . 23

8 Design decisions . . . 23

8.1 Asset management first, rent as a later module . . . 23

8.2 One role per level, with an assigned area . . . 23
8.3 Separation of duties in the registration chain . . . 23
8.4 404 instead of 403 outside the user's area . . . 23
8.5 Re-reading the user on every request . . . 23
8.6 No money on the public card or in the integration API . . . 24
8.7 A single server process . . . 24
8.8 Offline recording, online reading . . . 24
8.9 A read-only AI assistant that works with any provider . . . 24
9 Quality attributes . . . 24
9.1 Security . . . 24
9.2 Performance . . . 25
9.3 Availability and recovery . . . 25
9.4 Maintainability . . . 25
9.5 Usability and language . . . 25
10 Constraints, assumptions and open items . . . 25

1 Introduction

1.1 Purpose of this document

Jimma City Administration owns or controls thousands of assets: offices, market stalls and shops, land, vehicles, construction machinery, farms, halls and equipment. Until now the records for these assets have been kept on paper and in separate spreadsheets in each sector office. Nobody could say with confidence how many assets the City has, where they are, what condition they are in or who is responsible for them.

The Jimma City Smart Asset Management System (JCSAMS) replaces those scattered records with one register that every level of the City administration works on, from the sector office in a Kebele up to the Mayor. This document is the System and Architecture Design Document (SDD). It explains the architecture of that system, its key components, its data flows and the decisions that shaped it.

1.2 Intended readers

Reader	What they will use this document for
INSA assessment team	Understanding the attack surface: entry points, trust boundaries, where data is stored and how it moves (sections 3, 6, 7 and 9).
Jimma City ICT Directorate	Installing, operating and maintaining the system; planning the production server (section 5).
Developers who maintain the system	Finding where a function lives and how the parts depend on each other (sections 4 and 8).
Project sponsors at the Mayor's office	Checking that the design follows the City's chain of responsibility (sections 2 and 6).

1.3 Scope

The document covers the version of the system submitted for assessment (release 1.0, October 2026): the web application used by City staff, the REST API behind it, the public asset card opened by a printed QR label, the read-only integration API, the MySQL database, the file store for photos and documents, the scheduled jobs and the connections to e-mail and SMS services.

Rent billing, receipts and arrears management are planned for the next phase and are not part of this release. They will be built on the same asset register and are mentioned here only where the current design prepares for them.

1.4 Definitions and abbreviations

Term	Meaning
Kifle Ketema	Sub-city. Jimma has three: Hirmata, Jiren and Gibe.
Kebele	The smallest administrative unit. Each Kebele belongs to one Kifle Ketema. The system currently holds 13 Kebeles.
Sector	A line office of the City, for example Agriculture, Land, Trade or Construction. Each Kebele has the same 13 working sectors.
Asset ID	The unique, system-generated identifier of an asset, in the form AST-000125.
Public token	A random 16-character code printed in an asset's QR label. It opens the public asset card and reveals nothing by itself.

Term	Meaning
Area scope	The part of the City a user may see and act on: the whole City, one or more Kifle Ketemas, one Kebele, or one sector in one Kebele.
Approval state	Where a newly registered asset stands in the registration chain: DRAFT, SUBMITTED, REVIEWED, APPROVED, ACTIVE.
DFD	Data Flow Diagram (Gane and Sarson notation in this document).
JWT	JSON Web Token, the signed session token the browser sends with each request.
PWA	Progressive Web App: a web application that can be installed on a phone and opened without a connection.
SPA	Single-Page Application: the browser loads the application once and then exchanges only data with the server.
CSP	Content Security Policy, an HTTP header that tells the browser which scripts and resources it may load.

1.5 References

Reference	Title
JCSAMS-SRS-001	Software Requirements Specification, version 1.0
JCSAMS-TDD-003	Technical Design and Specification, version 1.0
JCSAMS-SEC-005	Security Documentation, version 1.0
ISO/IEC/IEEE 42010:2022	Systems and software engineering: architecture description
OWASP Top 10 (2021), OWASP API Security Top 10 (2023)	Web application and API security risk lists used for the security design
Project discussion record	Jimma City Asset Management System: Updated Discussion Summary and Conceptual Architecture, 6 October 2026

2 System overview

2.1 What the system does

The system keeps one record for every asset the City owns or controls and follows that asset through its whole life: acquisition, registration, approval, assignment to an office or person, use, inspection, maintenance, transfer, physical verification, retirement and disposal. Every change is kept with the name of the person who made it and the time, and nothing is overwritten without the earlier value being kept.

The main functions are:

- **Asset register.** Registration with a unique Asset ID, category and type, sector, location down to the Kebele and GPS point, ownership, responsible officer, condition, status, photos and documents.
- **Registration chain.** A Sector Administrator registers an asset; the Kebele Administrator confirms on site that it exists; the Kifle Ketema Administrator approves it. Only then does the asset become active.
- **Operations.** Maintenance, inspections (by Inspectors only), transfers, assignments, ownership changes, reservations of halls and vehicles, retirement and disposal, each with its own approval rules.
- **Physical verification.** Stock-taking campaigns in which officers scan QR labels and record whether each asset was found, missing or in the wrong place; differences are investigated and corrected with approval.

- **Money.** Acquisition cost, valuations, straight-line depreciation and income per asset. These figures are visible only to roles with the financial permission.
- **Monthly reports.** Each level sends a monthly report to the level directly above it. The system prepares the figures and freezes them when the report is sent.
- **Oversight.** Dashboards for each level, a map of assets, analytics, notifications by e-mail and SMS, and an audit log that only the Super Administrator and the Internal Auditor can read.
- **AI assistant.** A chat window on every page answers how-to questions and looks things up in the user's own area. It can only read.

2.2 Users and their areas

The system has eight roles. Five follow the City's line of command; three are specialist roles that sit beside it. A user only ever sees the part of the City that belongs to their role and assigned area. Table 1 summarises them; the full permission matrix is in the Security Documentation (JCSAMS-SEC-005).

Role	Main responsibility	Area seen
Super Administrator	Accounts of the Kifle Ketema Administrators and specialists, master data, settings, backups, audit log. Does no asset work.	Whole City
Mayor / Executive	Reads dashboards, analytics and money; receives the monthly reports of the Kifle Ketemas and the Internal Auditor.	Whole City
Kifle Ketema Administrator	Approves and activates assets and requests; manages the Kebele Administrators; follows the Inspectors.	One Kifle Ketema
Kebele Administrator	Verifies new assets on site; runs stock-taking; manages the Sector Administrators.	One Kebele
Sector Administrator	Registers and maintains the assets of one sector; raises requests.	One sector in one Kebele
Internal Auditor	Independent read-only review of records, money, users and the audit log.	Whole City
Financial Officer	Records valuations and income.	Assigned Kifle Ketemas
Inspector	The only role that records inspections; records verification results.	Assigned Kifle Ketemas

Table 1: Roles and areas

2.3 System context

Figure 1 places the system among the people and systems it exchanges data with. All staff use the same web application; what they can do is decided on the server by their role and area. The only part that needs no account is the public asset card, which a citizen sees when scanning a QR label with an ordinary phone camera. It answers the question "whose is this and is it registered?" and shows no money, no personal names and no exact position.

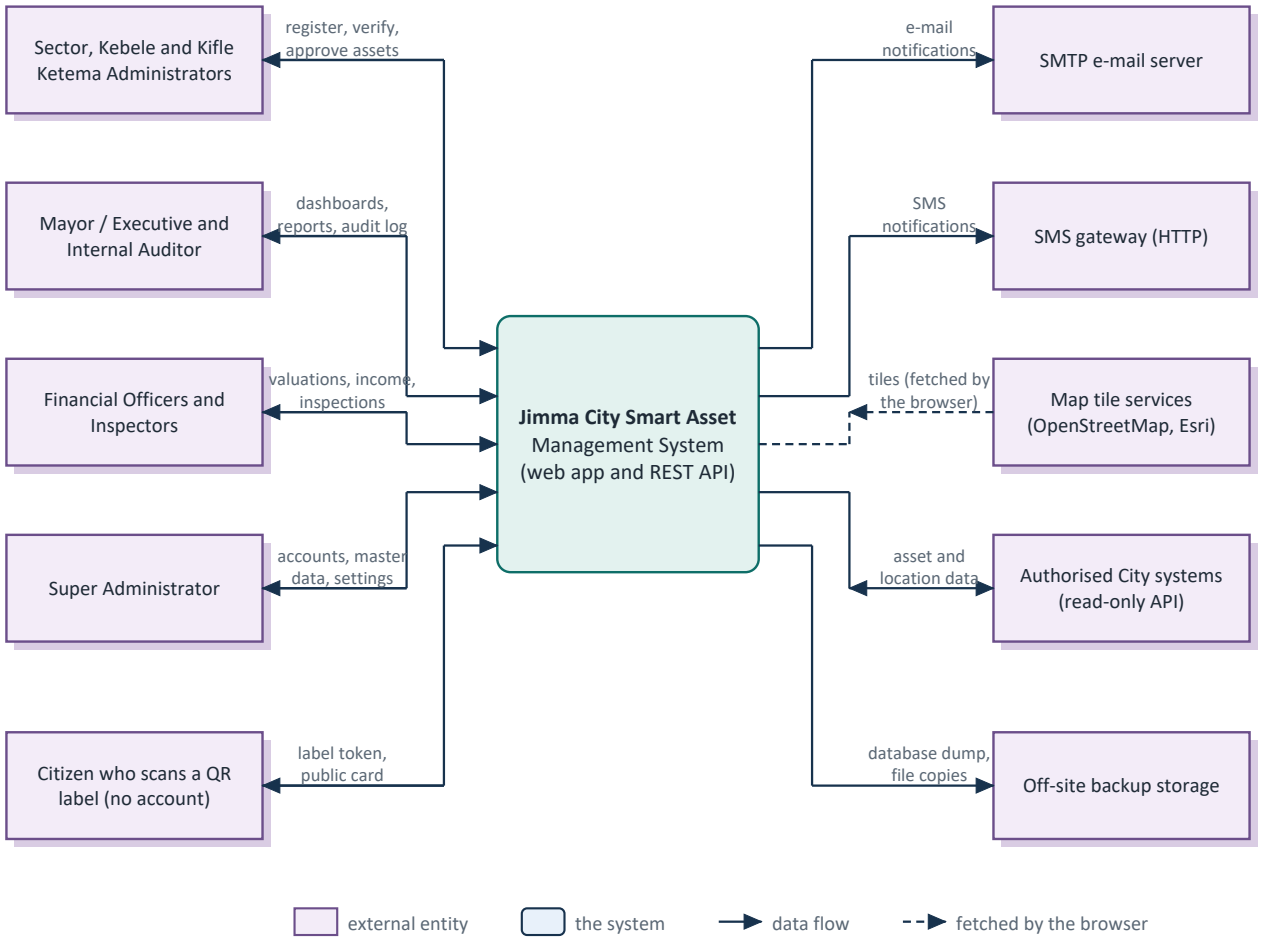


Figure 1: System context diagram

Outgoing connections are few and fixed: the SMTP server and the SMS gateway configured by the City, and, when the AI assistant is switched on, the AI service the City configures (section 6.7). The server never fetches a URL supplied by a user. Map tiles are loaded by the user's browser directly from OpenStreetMap and Esri; the server is not involved, and the Content Security Policy allows only those two image sources.

3 Architecture

3.1 Architectural style

JCSAMS is a three-tier web application with a single-page client. We chose this style deliberately and kept the number of moving parts small, because the system will be run by the City's own ICT staff on one server in the City data centre:

- **One application process.** A single Node.js process serves both the API and the built web application. There is no message broker, cache server or separate front-end server to secure and monitor. The design still allows a second instance behind the proxy later; the only in-memory state is the rate-limit counters.
- **A stateless API.** The server keeps no session in memory. Each request carries a signed token, and the server re-reads the user from the database on every request, so a deactivated account or a changed role takes effect immediately.
- **A relational database as the single source of truth.** Everything except the uploaded files lives in MySQL, with foreign keys, check constraints and transactions. Backups are one SQL dump plus one folder of files.
- **Authorisation on the server only.** The browser hides menu entries a user cannot use, but that is a convenience. Every request is checked again on the server for the role's permission and the user's area.

3.2 Architecture diagram

Figure 2 shows the tiers and the components inside each tier. Arrows show the direction in which requests are made.

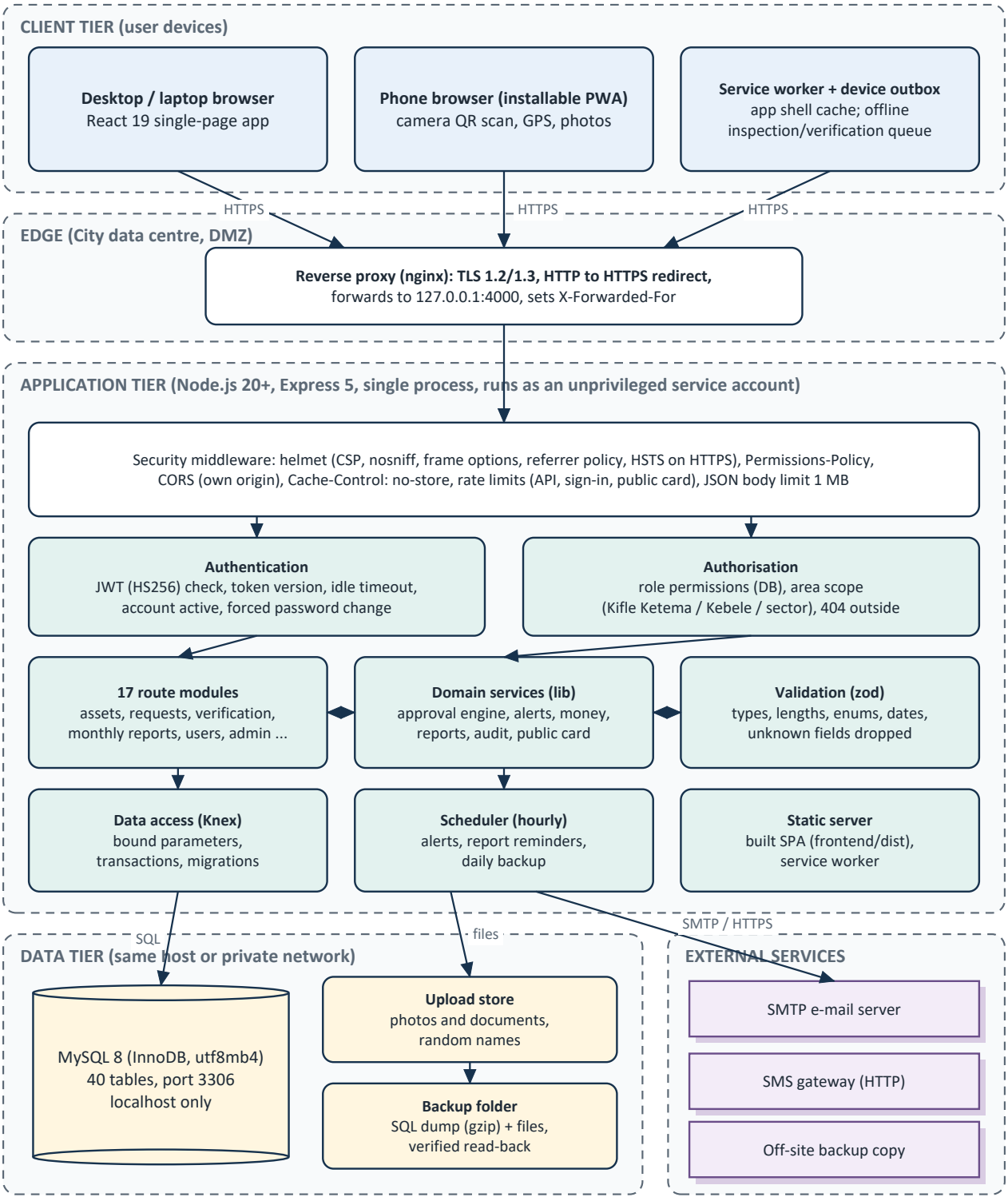


Figure 2: System architecture: tiers and components

3.3 Client tier

The client is a React 19 single-page application built with Vite into static files (HTML, JavaScript, CSS, fonts and images). It is served by the same Node.js process. The application talks to the server only through JSON requests under `/api`. The session token is kept in the browser's session storage, which is cleared when the browser is closed; it is never placed in a URL or a cookie.

On a phone the application can be installed to the home screen. A service worker caches only the application shell (the page, icons and built files) and never caches `/api` answers, because those contain personal and

financial data. Inspectors and Kebele staff working in the field can record inspections and verification results without a connection: the request is kept in an outbox on the device and sent automatically when the connection returns. The server applies the same checks to a delayed request as to any other.

The camera (for QR scanning and photos) and the location (for on-site checks) are the only device features the application uses. The server's Permissions-Policy header forbids the microphone, payment, USB, serial and Bluetooth APIs.

3.4 Application tier

The application tier is a Node.js (version 20 or later) process using the Express 5 framework. Its code is organised in layers; a request passes through them in the order of Figure 3.

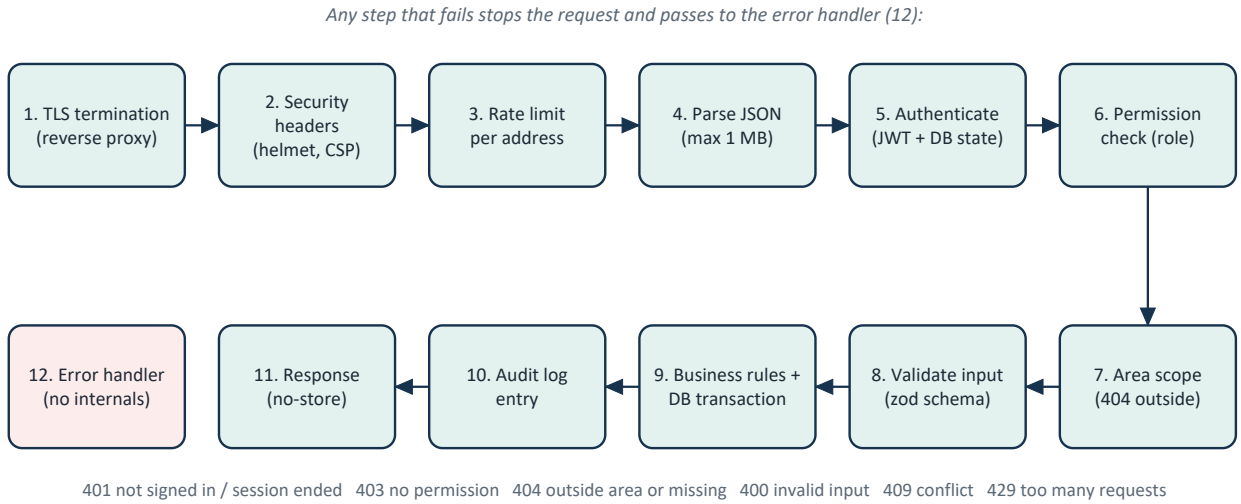


Figure 3: Processing of one API request

Step 5 does more than check the token signature. It loads the user from the database and refuses the request if the account is inactive, if the token's version no longer matches the user's (the user signed out, changed the password, or an administrator ended the sessions), if the user has been idle for more than 30 minutes, or if the user still holds a temporary password that must be replaced. Step 7 limits every query to the user's area; a record outside it answers 404 Not Found, so its existence is not revealed.

3.5 Data tier

MySQL 8 (or MariaDB 10.6 and later) with the InnoDB engine and the utf8mb4 character set holds 40 tables. The schema is created and upgraded only by numbered migration files (001 to 020 at the time of writing), so every installation has the same structure. Uploaded photos and documents are stored on disk under random file names, outside the web root, and are only served through an authenticated, scope-checked route. Backups are a compressed SQL dump plus a copy of the upload folder; each backup is read back after it is written to confirm it is usable.

4 Key components

Table 2 lists the components that make up the system, what each is responsible for, and where it lives in the source code. Paths are relative to the project root.

Component	Responsibility	Location
Web client (SPA)	All screens: dashboards, asset forms, approvals, verification, reports, administration. Three interface languages (English, Amharic, Afaan Oromo).	frontend/src
Service worker and outbox	Opens the app without a connection; keeps field results until the network returns.	frontend/public/sw.js, frontend/src/offline.js
HTTP server and middleware	Security headers, CORS, rate limits, body size limit, static files, error handler.	backend/src/app.js, server.js
Authentication	Sign-in, account lockout, sessions, sign-out, password change, temporary passwords.	backend/src/routes/auth.js, middleware/auth.js, lib/security.js
Authorisation and scope	Role permissions (editable by the Super Administrator), area scope, who may manage which accounts.	backend/src/lib/permissions.js, lib/assetGuards.js
Asset register	Create, read, update, search, export; registration chain; photos and documents.	backend/src/routes/assets.js, lib/assetQuery.js
Asset operations	Acquisition, assignment, ownership change, disposal, QR codes and labels, scanning, bulk import, duplicates.	backend/src/routes/assetOps.js
Approval engine	Requests for transfer, disposal, retirement, assignment and ownership change; one or two steps; separation of duties.	backend/src/lib/requests.js, routes/requests.js
Physical verification	Campaigns, results, discrepancies, corrections, CSV report.	backend/src/routes/verification.js
Finance	Valuations, income, depreciation, finance overview.	backend/src/lib/money.js, lib/depreciation.js, routes/finance.js
Monthly reports	Preparing frozen figures, sending up the chain, accepting or returning.	backend/src/lib/monthlyReports.js, routes/monthlyReports.js
Alerts and notifications	21 notification types; in-app always, e-mail and SMS when configured.	backend/src/lib/alerts.js, notify.js, channels.js
Scheduler	Hourly run of alerts and report reminders; optional daily backup.	backend/src/lib/scheduler.js
Audit trail	Insert-only log of every change, sign-in, download and export.	backend/src/lib/audit.js, routes/audit.js
Public asset card	The page a printed QR label opens without signing in.	backend/src/lib/publicAsset.js, routes/public.js
Integration API	Read-only REST API for other City systems, with API keys and an OpenAPI description.	backend/src/routes/integration.js
AI assistant	Chat window; server-side instructions; four read-only lookups run with the user's scope; any OpenAI-compatible AI service.	frontend/src/components/Assistant.jsx, backend/src/routes/assistant.js, lib/assistant.js
Administration	Workflows, permissions, settings, system health, backups, API keys.	backend/src/routes/admin.js, lib/backup.js
Data access and schema	Knex query builder, connection pool, migrations.	backend/src/db.js, src/migrations

Component	Responsibility	Location
Translations	Dictionaries for Amharic and Afaan Oromo, served to the client.	backend/src/i18n

Table 2: Key components

4.1 Component dependencies

Dependencies run in one direction: route modules call domain services, and domain services call the data-access layer. No domain service calls a route, and the client never reaches the database or the file store except through the API. Cross-cutting rules have a single home so that they cannot drift apart: permissions and scope in `permissions.js`, input rules in `schemas.js` and `assetSchemas.js`, money rules in `money.js`, and the audit entry in `audit.js`.

4.2 Third-party components

Package	Version	Used for
express	5.2	HTTP server and routing
helmet	8.3	Security headers including the Content Security Policy
jsonwebtoken	9.0	Signing and verifying session tokens (HS256)
bcryptjs	3.0	Password hashing, cost factor 12
knex, mysql2	3.3, 3.24	Query builder with bound parameters; MySQL driver
zod	4.6	Validation of every request body and query
multer	2.4	Receiving uploads (one file, 10 MB limit)
nodemailer	10.0	Sending e-mail through the configured SMTP server
qrcode	1.5	Drawing QR codes for asset labels
csv-parse	7.0	Reading bulk-import files
react, react-dom, react-router-dom	19.3, 7.18	User interface and client-side routing
leaflet	1.9	The asset map
html5-qrcode, jsbarcode	2.3, 3.12	Camera QR scanning; Code 128 barcodes on labels

All versions are fixed in `package-lock.json`. On 25 October 2026 `npm audit` reported no known vulnerabilities in either the backend or the frontend.

5 Deployment architecture

The system is designed to run on one Linux server in the City data centre. Figure 4 shows the recommended production layout. Only the reverse proxy is reachable from the network; the Node.js process and MySQL listen on the loopback interface.

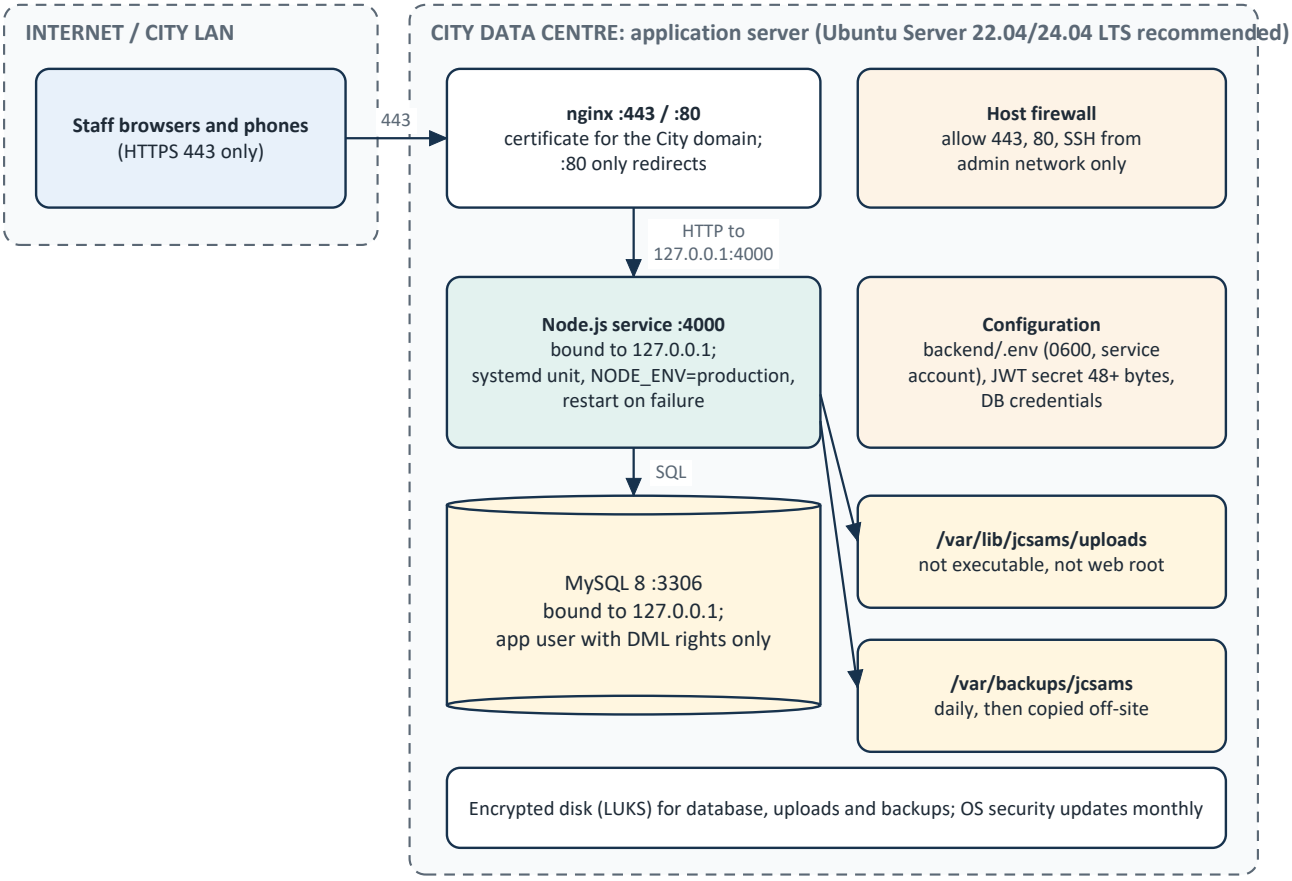


Figure 4: Production deployment

Item	Minimum	Recommended
Processor	2 cores	4 cores
Memory	4 GB	8 GB
Disk	100 GB	250 GB SSD, encrypted, plus a separate disk or share for backups
Operating system	Any 64-bit Linux with Node.js 20	Ubuntu Server 22.04 or 24.04 LTS
Software	Node.js 20+, MySQL 8 or MariaDB 10.6+, nginx	Same, installed from the distribution's security-supported packages
Network	One public or LAN address, ports 80 and 443	City domain name with a valid TLS certificate

Table 3: Server requirements

The environment used for the INSA assessment follows the same layout. Its address, the test accounts for each role and the test window are given in the covering letter (JCSAMS-LTR-000) and agreed with the assessment team before testing starts.

6 Data flow design

This section describes how data enters the system, where it is processed and stored, and where it leaves. The diagrams use the Gane and Sarson notation: rectangles with a shadow are external entities, rounded boxes are processes, and open-ended rectangles are data stores.

6.1 Context level (level 0)

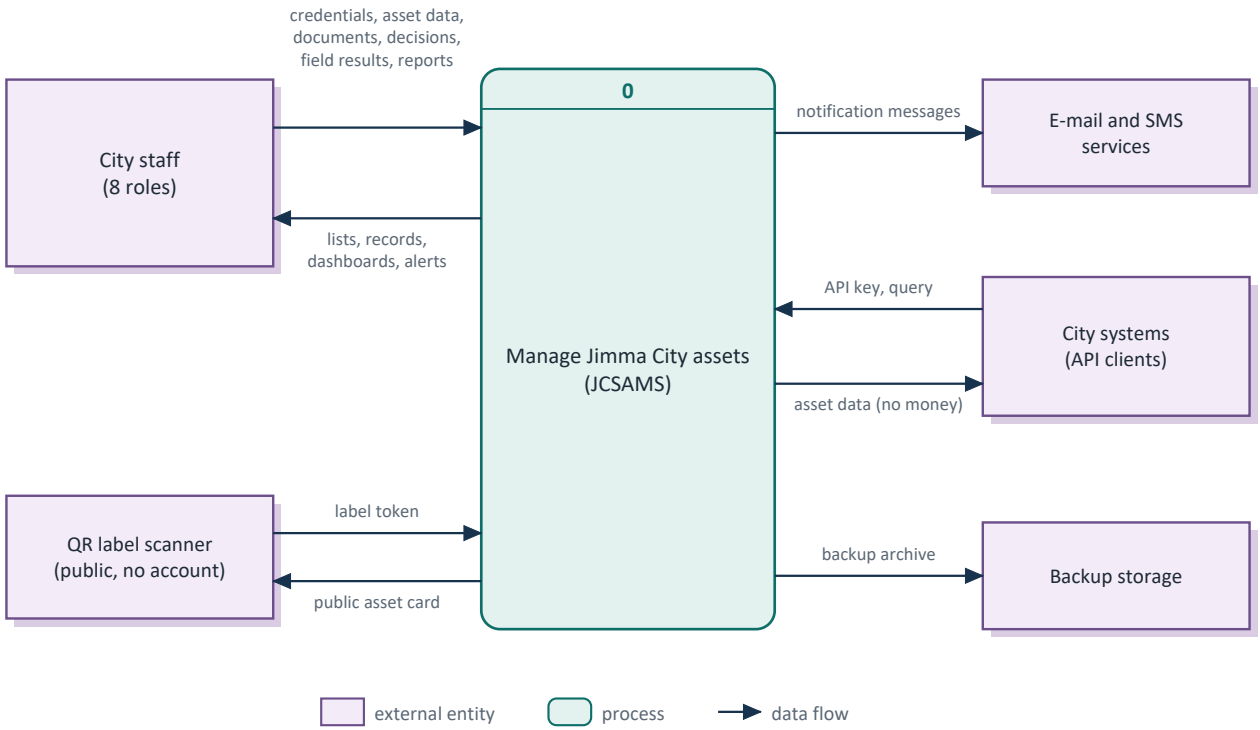


Figure 5: Data flow diagram, level 0

At the highest level the system receives credentials, asset data, documents, decisions and field results from City staff and returns lists, records, dashboards and alerts. Anonymous visitors send only a label token and receive the public card. City systems send an API key and a query and receive asset, location and classification data without any money figures. The system sends notification messages to the e-mail and SMS services and backup archives to the backup storage.

6.2 Main processes (level 1)

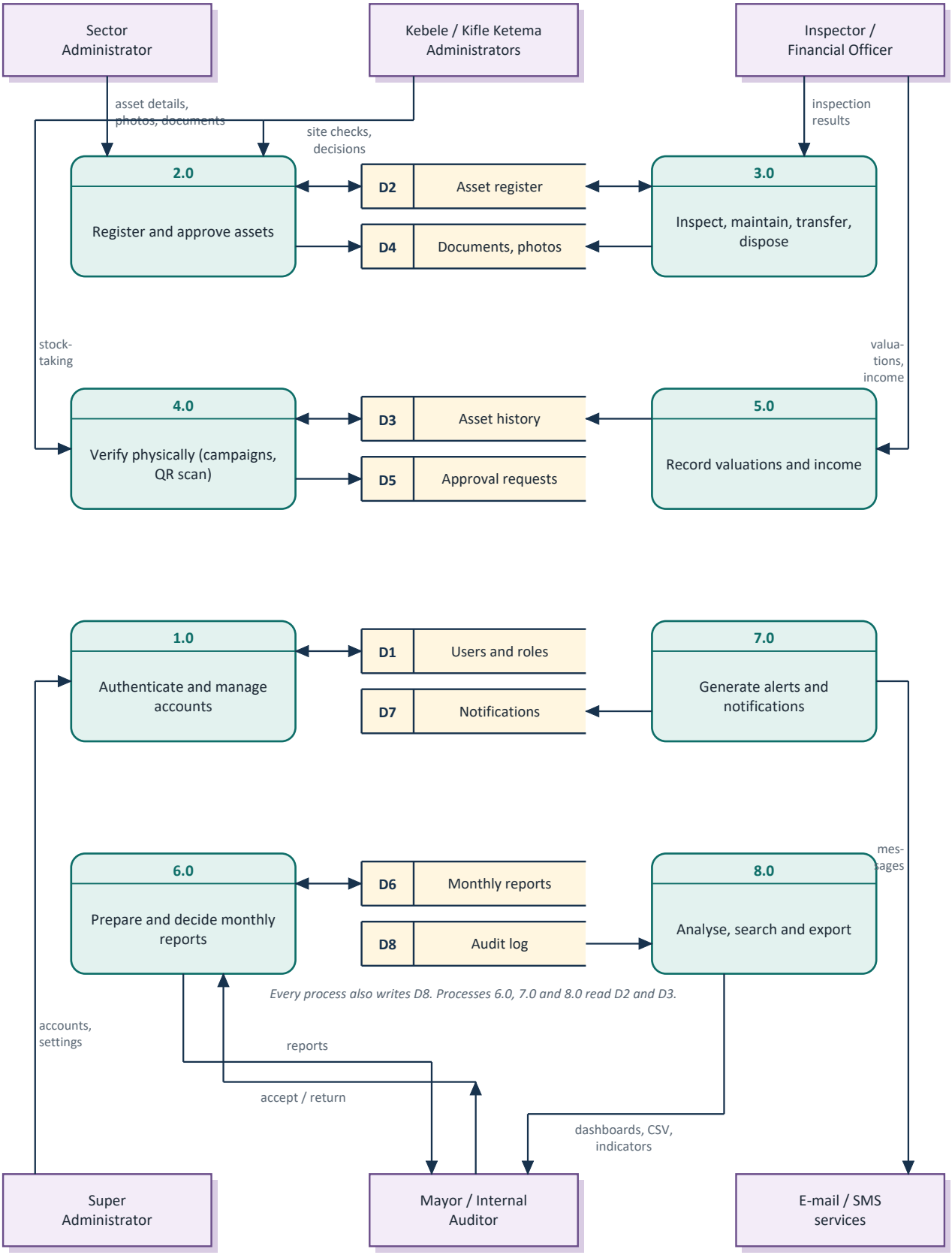


Figure 6: Data flow diagram, level 1

Figure 6 breaks the system into eight processes and eight data stores. The stores are logical groupings of the 40 physical tables; the mapping is in Table 4.

Store	Holds	Physical tables
D1 Users, roles, permissions	Accounts, password hashes, lock state, areas, role permissions, API keys	users, user_kifle_ketemas, user_kebeles, user_sectors, role_permissions, api_keys
D2 Asset register	The current record of each asset and the master lists	assets, asset_categories, asset_types, sectors, departments, kifle_ketemas, kebeles
D3 Asset history records	Everything that happened to an asset	maintenance_records, inspection_records, asset_transfers, asset_assignments, asset_ownership_history, asset_valuations, asset_revenues, asset_acquisitions, asset_disposals, utilizations, asset_reservations, asset_site_checks, follow_up_actions, scan_events, verification_campaigns, verification_items, asset_retired_labels
D4 Documents and photos	File metadata in the database; the files on disk	asset_documents, upload folder
D5 Approval requests	Requests and their decisions	approval_requests, workflow_configs
D6 Monthly reports	Reports with their frozen figures	monthly_reports, report_schedules
D7 Notifications	Messages per user and their delivery status	notifications, notification_rules
D8 Audit log	Who did what, when, with old and new values	audit_logs

Table 4: Logical data stores and physical tables

Table 5 shows which process creates (C), reads (R), updates (U) or deletes (D) data in each store. A process that is not allowed to touch a store has an empty cell; this is the same rule the server enforces through permissions.

Process	D1	D2	D3	D4	D5	D6	D7	D8
1.0 Authenticate and manage accounts	CRU	R					C	C
2.0 Register and approve assets	R	CRUD*	C	CRUD*			C	C
3.0 Inspect, maintain, transfer, dispose	R	RU	CRU	C	CRU		C	C
4.0 Verify physically	R	RU	CRU	C			C	C
5.0 Record valuations and income	R	RU	C					C
6.0 Prepare and decide monthly reports	R	R	R		R	CRU	C	C
7.0 Generate alerts and notifications	R	R	R	R	R	R	CRU	
8.0 Analyse, search and export	R	R	R	R	R			C

*Table 5: Process and data store access (C create, R read, U update, D delete). * Delete only for a draft that was never submitted, by the person who created it.*

6.3 Asset registration (level 2 of process 2.0)

Registration is the flow that matters most for the integrity of the register, so it is drawn in more detail in Figure 7. Each step is taken by a different person, and the system refuses a step from someone who has already taken an earlier step on the same asset.

Dashed: a returned asset goes back to DRAFT with a reason, and the next person is notified (D7). Every step is written to D8.

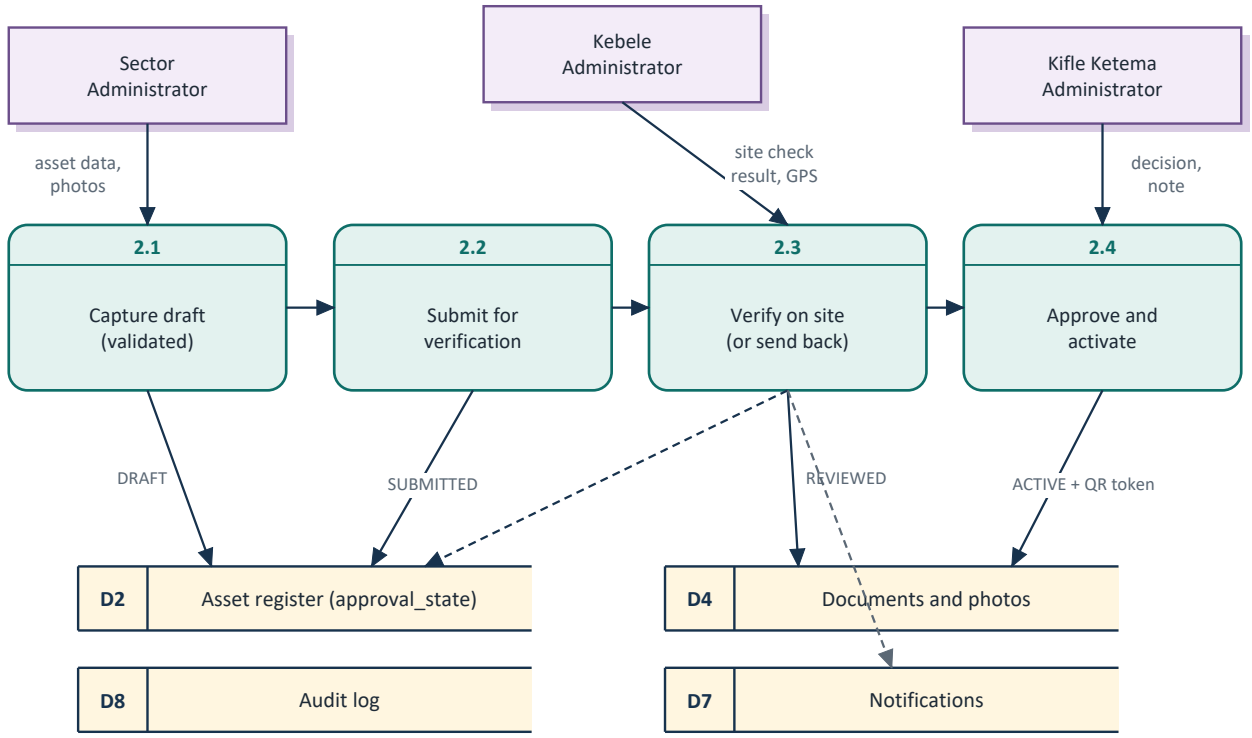


Figure 7: Data flow diagram, level 2: asset registration and approval

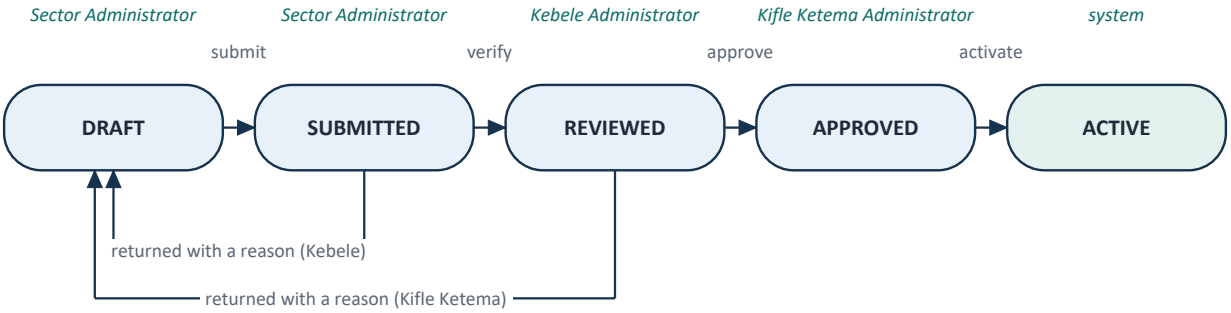


Figure 8: Approval states of a newly registered asset

6.4 Data flow explanation

Table 6 describes each important data flow: what is carried, between which parts, and how it is protected. The identifiers are used in the Security Documentation when describing threats to each flow.

ID	Flow	Data carried	Protection
F1	Staff browser to API: sign-in	Username and password	TLS; rate limit 20 per minute per address; account lock after 5 failures; same answer and timing for unknown users; password never logged.
F2	API to browser: session	Signed JWT (user id, token version, 8-hour expiry)	TLS; kept in session storage only; no-store caching; invalidated on sign-out, password change, role change or deactivation.
F3	Browser to API: asset registration	Asset details, GPS point, photos, documents	Role and area check; schema validation; file content check by signature; 10 MB limit; random stored name; audit entry.
F4	Kebele Administrator to API: site check	Found / not found, notes, GPS, photo	Only for assets of their own Kebele; refused if the same person registered the asset.
F5	Kifle Ketema Administrator to API: decision	Approve or return, reason	Only for their own Kifle Ketema; refused if the same person reviewed it; state change and audit entry in one transaction.
F6	Inspector / Financial Officer to API	Inspection results; valuations and income	Only Inspectors may record inspections; only Financial Officers may record money; area of the assigned Kifle Ketemas only; dates cannot be in the future.
F7	Field device outbox to API	Delayed inspection and verification results	Sent with the user's current token; the same checks as F6; refused results stay on the device and are shown to the user.
F8	API to database	Queries and changes	Bound parameters through Knex; the application database user has data rights only; MySQL on localhost.
F9	API to file store	Photos and documents	Outside the web root; served only through an authenticated route that checks the user's area; downloads are logged and sent as attachments.
F10	Visitor to public card	Label token; public card	Rate limit 120 per minute; token pattern check; only safe fields returned; replaced labels answer "label replaced".
F11	API client to integration API	API key; asset, location and classification data	Key stored only as a SHA-256 hash; read-only; no money figures; key can be disabled at once.
F12	API to e-mail / SMS service	Notification text, temporary passwords for new accounts	SMTP with TLS where the server offers it; the server log records only the recipient and message length, never the text.
F13	Sender to receiver: monthly report	Frozen figures, comments, decision	Visible only to the sender and the direct superior; accepted reports are locked; money only for roles that may see it.
F14	Server to backup storage	SQL dump and file copy	Written by the server, read back to verify; copying off-site is an operator task on an encrypted medium.
F15	Server to AI service and back	The question, the server's instructions, results of read-only lookups; the answer	Off unless configured; HTTPS to cloud services or loopback to a local model; lookups scoped to the user and money masked; no write functions; 10 questions/min per person; audit entry without the question text. Free cloud tiers may keep the data: City decision required (SRS OI-8).

Table 6: Data flows and their protection

6.5 Sequence of a sign-in and an authenticated request

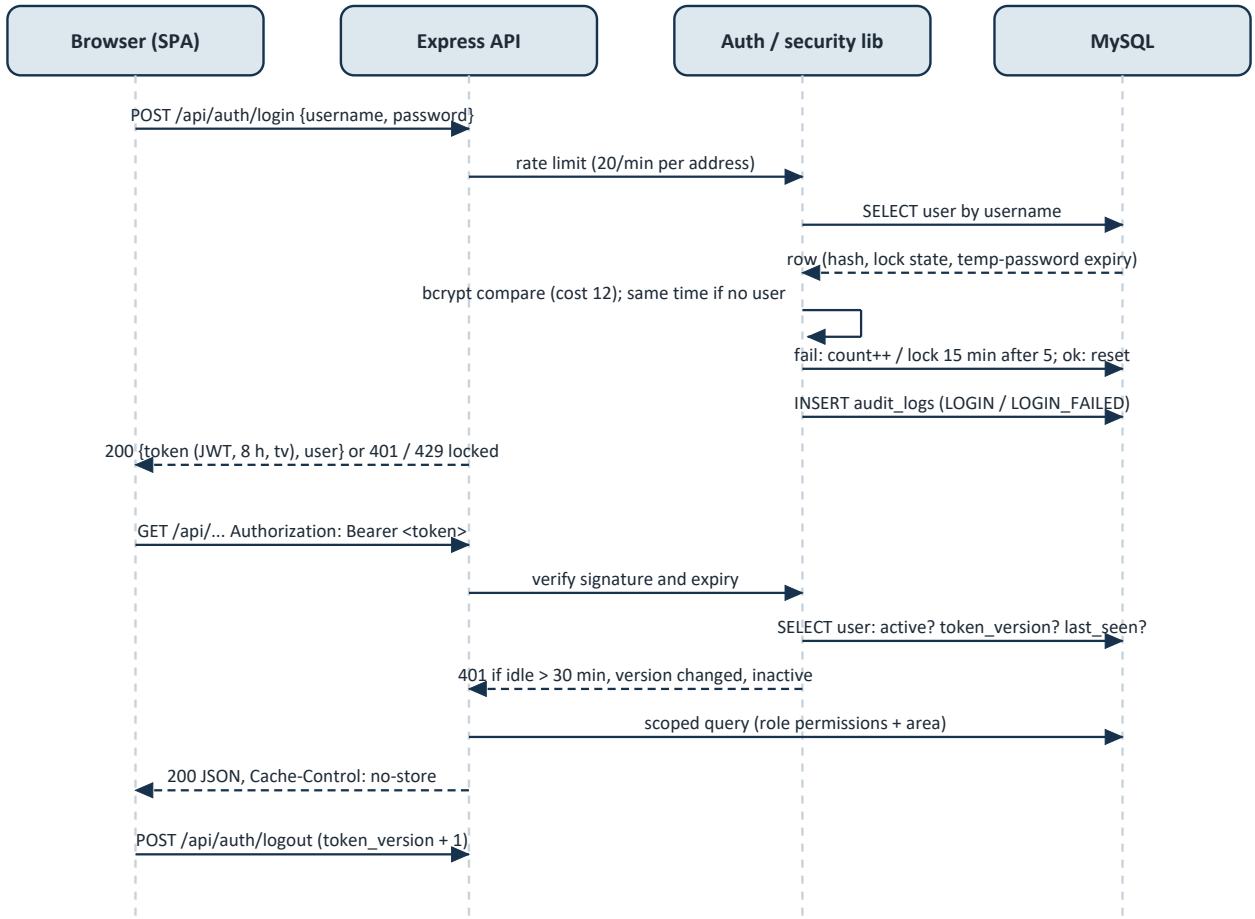


Figure 9: Sign-in, an authenticated request and sign-out

6.6 Sequence of a QR label scan

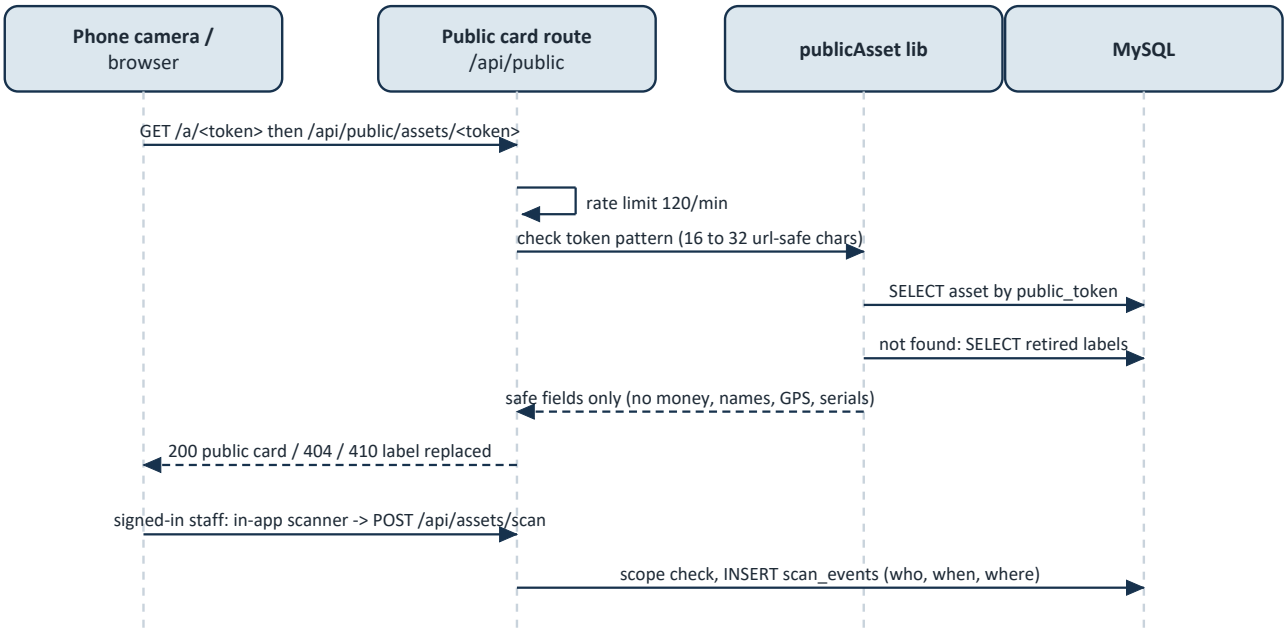


Figure 10: Public QR card and in-app scanning

6.7 AI assistant

The assistant is part of process 8.0 (analyse, search and export). The browser sends only the conversation; the server adds its own instructions, a short guide to the system and four lookup functions, and sends them to the configured AI service. When the model asks for a lookup, the server runs it as the signed-in user, through the same scoped queries the screens use, and returns the result to the model as data. The model never reaches the database and has no function that writes. Figure 11 shows one question.

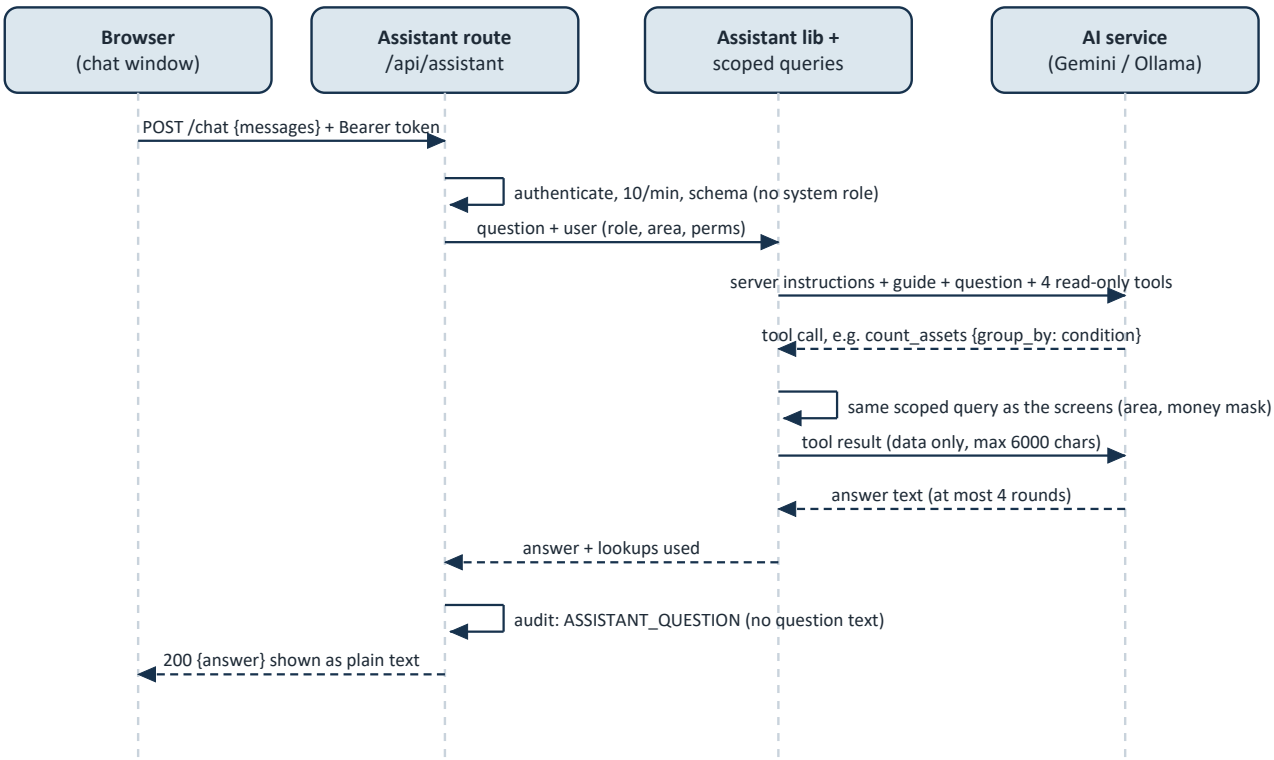


Figure 11: One question to the AI assistant

7 Data design

7.1 Core entities and relationships

Figure 12 shows the core entities. The asset sits at the centre; everything that happens to it is a separate child record, so the asset row holds the current state and the child tables hold the history. Location is modelled as a hierarchy (Kifle Ketema, Kebele) and is kept separate from classification (category, type) and from organisation (sector, department).

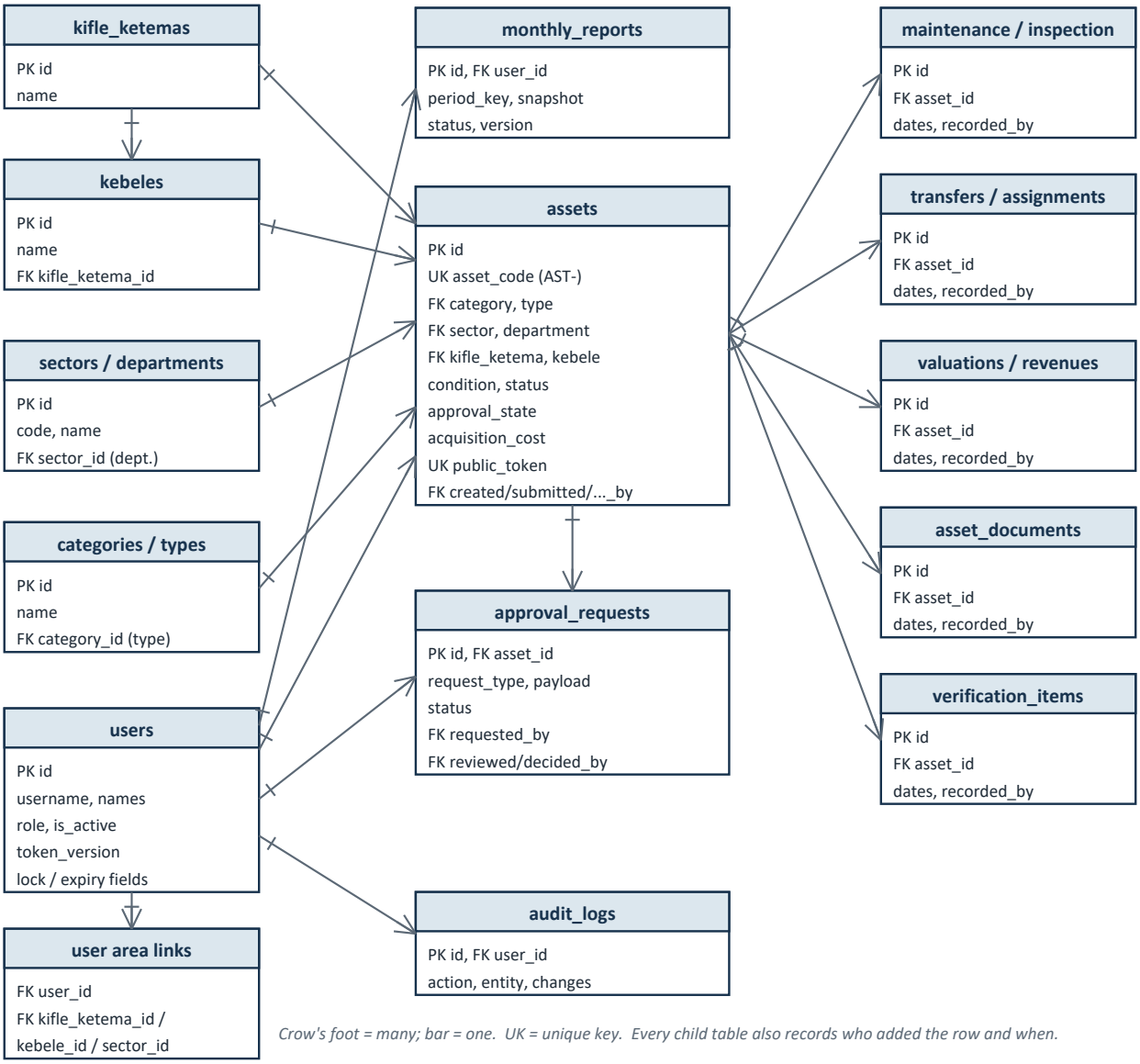


Figure 12: Core entity-relationship diagram (simplified)

7.2 Design rules for data

- **One asset, one record.** An asset is never duplicated because its use or its revenue changes. Use is recorded in utilizations; income is recorded against the asset with its type.
- **Nothing is silently overwritten.** Transfers, assignments, ownership changes, valuations, condition and status keep every earlier value in their history tables.
- **The audit log is insert-only.** The application has no code path that updates or deletes an audit entry, and the API offers no such operation.
- **Constraints in the database as well as in the code.** Foreign keys, unique keys (Asset ID, username, public token) and check constraints (for example non-negative money) are defined in the migrations, so bad data is refused even if a code check were missed.
- **Disposed assets are locked.** After disposal the record can be read and archived but not changed.
- **Working labels are marked.** Master data that the City has not yet confirmed officially (some sectors and categories) is flagged and shown as a working label.

7.3 Data classification

Class	Examples	Who can see it
Public	Asset name, category, sector, Kebele, condition, status, photos (public card only)	Anyone holding the asset's label token
Internal	Full asset record, history, documents other than financial ones	Staff whose area covers the asset
Confidential: financial	Acquisition cost, valuations, depreciation, income; purchase, invoice and contract documents	Roles with the financial permission, within their area
Confidential: personal	Names, phone numbers, e-mail addresses, profile photos of staff	The account's manager in the hierarchy, the Internal Auditor (read-only), the person themselves
Secret	Password hashes, JWT secret, API key hashes, database credentials	Never returned by the API; held only in the database or the server configuration

8 Design decisions

The decisions below shaped the system. Each was discussed with the City project team; the reason is recorded so that a later change can be judged against it.

8.1 Asset management first, rent as a later module

The City first proposed a rent revenue system. We found that rent is one use of an asset, and that many assets (an ambulance, an office) earn nothing. Building the asset register first means rent, agriculture and other revenue modules can later use the same asset identity instead of each keeping its own copy.

8.2 One role per level, with an assigned area

The 13 Kebeles do not become 13 roles. There is one Kebele Administrator role, and each account is assigned one Kebele. Adding a Kebele is data, not code.

8.3 Separation of duties in the registration chain

The person who registers an asset cannot verify it, and the person who verifies it cannot approve it. This prevents a single person from creating a fictitious asset or hiding a missing one.

8.4 404 instead of 403 outside the user's area

If a Kebele Administrator asks for an asset in another Kebele, the answer is "not found". A "forbidden" answer would confirm that the asset exists, which is itself information.

8.5 Re-reading the user on every request

This costs one indexed query per request but means a deactivated account, a role change or a changed permission takes effect immediately rather than when the token expires.

8.6 No money on the public card or in the integration API

Values and income are confidential. Other City systems that need them must ask for a separate, agreed interface.

8.7 A single server process

The City's ICT team will operate the system. A single process with MySQL is something they can run, back up and restore themselves. The cost is that rate-limit counters live in memory; this is documented and acceptable for one server.

8.8 Offline recording, online reading

Field staff in parts of the City with weak coverage must still be able to record inspections. Reading the register offline would mean storing personal and financial data on phones, so it was deliberately left out.

8.9 A read-only AI assistant that works with any provider

The assistant helps staff who are new to computers, in their own language. It is read-only and runs every lookup as the user, so it cannot become a way around the permissions. It uses the common OpenAI-compatible interface so the City can choose a free cloud service now and a model on its own server later, without code changes.

9 Quality attributes

9.1 Security

Security is described fully in JCSAMS-SEC-005. Figure 13 shows the trust boundaries the design relies on: every request crosses TB1 encrypted, only the proxy crosses TB2, and only the application crosses TB3.

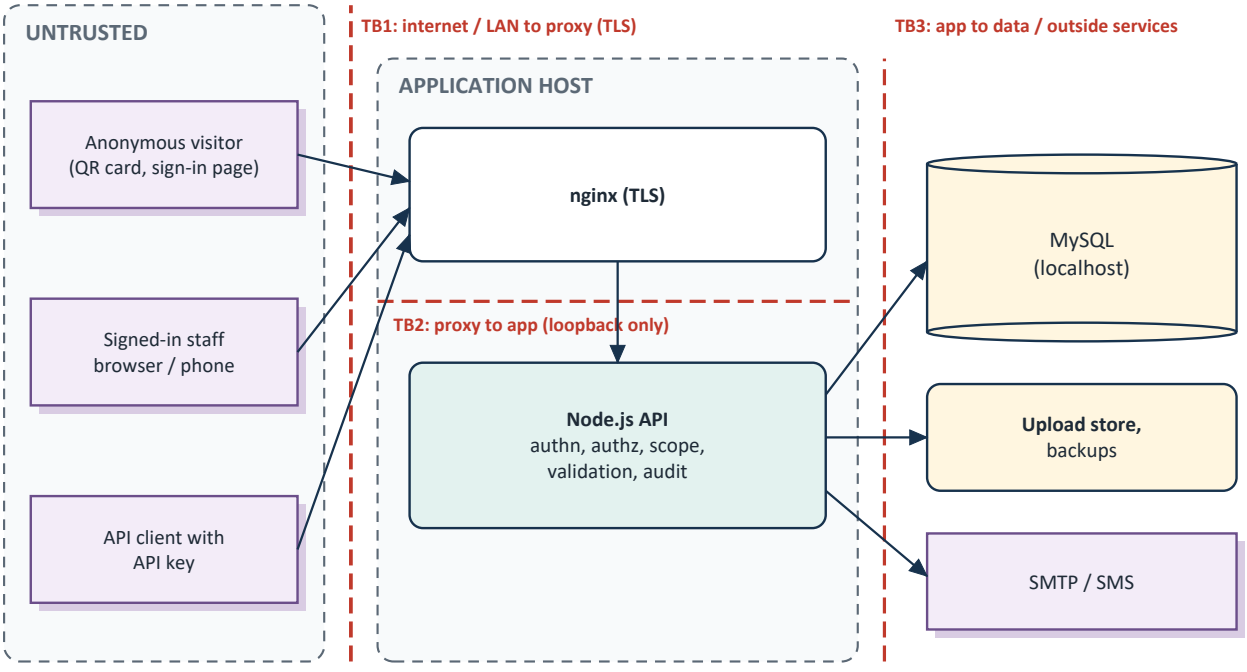


Figure 13: Trust boundaries

9.2 Performance

Lists are paged on the server and every list query is a single SQL statement with joins; no list issues one query per row. Columns used for filtering and scope (Kebele, Kifle Ketema, sector, status, approval state, asset code, public token) are indexed. The system has not yet been load-tested with the City's full volume of assets, photos and history; that test is planned before go-live and is listed as an open item in the SRS.

9.3 Availability and recovery

The Node.js service is restarted automatically by systemd if it stops. A database and file backup can be taken on demand or daily; each backup is verified by reading it back, and the administration screen warns when the last good backup is too old. A restore script rebuilds the database from a backup. The recovery point is at most one day with daily backups; the recovery time on the recommended server is under one hour for the expected data size.

9.4 Maintainability

The backend has 265 automated tests in 16 files that run against a separate test database. They cover sign-in and lockout, access by role and area, the approval chain and separation of duties, validation, uploads, monthly reports, verification, notifications, backups and the public card. All 265 passed on 25 October 2026. Schema changes are made only through numbered migrations, and translations are kept in plain JSON dictionaries that the City's language staff can edit.

9.5 Usability and language

The interface is available in English, Amharic and Afaan Oromo, chosen at sign-in or from the menu. Monthly reports follow the Ethiopian calendar by default. The layout adapts to phone screens, and the forms check each field as it is typed (names accept only letters, phone numbers only digits up to their length, money only valid amounts).

10 Constraints, assumptions and open items

#	Item	Status
1	Which Kifle Ketema each of the 13 Kebeles belongs to must be confirmed by the City. Until it is, assets cannot be registered in an unassigned Kebele.	Open: City to confirm
2	The official list of asset categories and revenue classifications is still a working list.	Open: City to confirm
3	Production requires HTTPS with a certificate for the City's domain; without it, phone cameras and location will not work.	Planned for deployment
4	E-mail and SMS delivery depend on the City providing an SMTP server and an SMS gateway.	Open: City ICT
5	Load testing with the full City data volume.	Planned before go-live
6	Two-factor sign-in and single sign-on with other City systems are not in this release.	Next phase
7	The Amharic and Afaan Oromo translations should be reviewed by the City's language staff before official use.	Open: City